

Python Full Stack Assignment - Part 2

CodeHub: Backend Data Validation & Processing System

Scenario

The CodeHub application has grown.

In Part 1, you built the **User Registration System**.

Now the backend receives messy and inconsistent user data from different sources:

- Registration forms
- CSV files
- Admin panels
- API requests
- Imported student records

Your job is to build a **Data Validation & Processing System** that cleans the incoming data and checks whether it is valid.

This assignment focuses on the remaining Python string methods.

Methods you MUST use

`splitlines()` `partition()` `replace()` `isalpha()` `isdigit()` `isalnum()` `isspace()` `isupper()`
`islower()` `istitle()` `isnumeric()` `center()` `ljust()` `rjust()` `zfill()`

You may also use the methods from **Part 1** such as: `strip()`, `split()`, `join()`, `upper()`, `lower()`, `title()`, `find()`, `count()`, `startswith()`, `endswith()`, etc.

Do not use

- Loops
- Functions
- Classes
- Regular expressions
- Exception handling
- External libraries

Use only the concepts you have learned so far.

Part 1 - Clean a User's Name

The application receives:

```
"john123"
```

The system needs to determine whether the name contains only alphabets.

Use:

```
name.isalpha()
```

Expected:

```
Is valid name: False
```

Test the following values:

```
John
JohnDoe
John123
John Doe
John_Doe
```

Think carefully about why JohnDoe is valid but John Doe is not.

Part 2 - Phone Number Validator

The application receives a phone number as a string.

Example:

```
phone = "9876543210"
```

Check whether the value contains only digits. Use:

```
phone.isdigit()
```

Display:

```
Phone Number : 9876543210
Valid         : True
```

Now test:

```
9876543210
98765abc10
98765 43210
+919876543210
```

Challenge

Explain why "9876543210" returns True but "+919876543210" returns False.

Part 3 - Student ID Validator

CodeHub assigns every student an ID.

Example:

```
student_id = "PYTHON2026"
```

Check whether the ID contains only:

- alphabets
- numbers

Use:

```
student_id.isalnum()
```

Test:

```
PYTHON2026
PYTHON_2026
PYTHON-2026
PYTHON 2026
```

Expected:

```
PYTHON2026 -> True
PYTHON_2026 -> False
PYTHON-2026 -> False
PYTHON 2026 -> False
```

Question

Why does `isalnum()` reject `_`, `-`, and spaces?

Part 4 - Password Validation

The CodeHub registration system needs to perform some basic password checks.

Given:

```
password = "Python123"
```

Check:

```
Is uppercase?
Is lowercase?
Is alphanumeric?
```

Use:

```
password.isupper()
password.islower()
password.isalnum()
```

Display:

```
Password : Python123

All uppercase : False
All lowercase : False
Alphanumeric  : True
```

Now test:

```
PYTHON
python
Python
Python123
Python@123
```

Think About It

Why does `"Python".isupper()` return `False`? And why does `"Python123".isalnum()` return `True`?

Part 5 - Username Case Checker

The admin dashboard wants to know how usernames are stored.

Given:

```
username = "john_doe"
```

Check whether the username is completely lowercase. Use:

```
username.islower()
```

Test:

```
john_doe
JOHN_DOE
John_Doe
john123
```

Display:

```
Username      : john_doe
Lowercase     : True
```

Part 6 - Student Name Format Checker

The database requires student names to be stored in title case.

Given:

```
student_name = "John Doe"
```

Use:

```
student_name.istitle()
```

Test:

```
John Doe  
john doe  
JOHN DOE  
John doe  
John123
```

Display:

```
Student Name : John Doe  
Correct Format : True
```

Bonus

If the name isn't correctly formatted, use `.title()` to fix it. For example, `john doe` becomes `John Doe`.

Part 7 - Numeric Data Detector

CodeHub receives different types of numeric-looking values.

Test:

```
"12345"  
"12.50"  
"-500"  
"■"  
"123abc"
```

Use:

```
.isnumeric()
```

Determine which values are considered numeric strings.

Important

Compare `.isdigit()` and `.isnumeric()` using:

```
"123"  
"■"
```

and observe the difference.

Part 8 - Empty Input Detector

A user accidentally submits:

```
address = "    "
```

Use:

```
address.isspace()
```

to determine whether the input contains only whitespace.

Display:

```
Address received : "    "
```

Contains only spaces : True

Now test:

```
"    "  
"hello"  
" hello "  
""
```

Think About It

Why does `"".isspace()` return False?

Part 9 - Process an Address

The application receives:

```
address = "Hyderabad:Telangana:India"
```

Use:

```
.partition(":")
```

to separate the address into three parts.

Expected:

```
('Hyderabad', ':', 'Telangana:India')
```

Now try `address = "Hyderabad:Telangana"` and `address = "Hyderabad"`. Observe what happens when the separator is missing.

Part 10 - Parse API Data

Imagine an API sends user information as one string:

```
data = "name:John Doe,email:john@gmail.com,city:Hyderabad"
```

Use `.partition()` to extract the first key-value pair. For example, `name:John Doe` should become:

```
Key   : name  
Value : John Doe
```

Challenge

Try to process `email:john@gmail.com` and `city:Hyderabad` using the same concept.

Part 11 - Clean Imported Data Using ``replace()``

An old system stores phone numbers like this:

```
"987-654-3210"
```

Remove the hyphens using:

```
phone.replace("-", "")
```

Expected:

```
9876543210
```

Now clean `"987 654 3210"` using `.replace()`.

Part 12 - Clean User Input

The application receives:

```
name = "John@Doe"
```

Replace @ with a space.

Expected:

```
John Doe
```

Use:

```
name.replace("@", " ")
```

Now try "Python#Full#Stack" and convert it to Python Full Stack.

Part 13 - Generate a URL Slug

CodeHub wants to generate URLs from course names.

Given:

```
course = "Python Full Stack Development"
```

Generate:

```
python-full-stack-development
```

You can combine `.lower()`, `.replace()`, and other methods you've already learned.

Example

```
Course Name : Python Full Stack Development
URL Slug    : python-full-stack-development
```

Part 14 - Process Multi-Line Data

The backend receives multiple student records in one string:

```
students = """John Doe
Alice Smith
Rahul Kumar
Sneha Reddy"""
```

Use:

```
.splitlines()
```

to separate the records.

Expected:

```
John Doe
Alice Smith
Rahul Kumar
Sneha Reddy
```

Challenge

Use `.splitlines()` and `.join()` to convert the data into:

```
John Doe | Alice Smith | Rahul Kumar | Sneha Reddy
```

Part 15 - Course Description Processor

CodeHub stores course descriptions like this:

```
description = """Python
Django
```

```
React
MySQL
AWS"""
```

Use `.splitlines()` to process the course modules. Then create a single string:

```
Python -&gt; Django -&gt; React -&gt; MySQL -&gt; AWS
```

Hint

You already know `.join()`.

Part 16 - Student Registration Number

Every student needs a 5-digit registration number.

Suppose:

```
number = "42"
```

Use:

```
.zfill(5)
```

Expected:

```
00042
```

Test:

```
"1"
"42"
"123"
"9999"
"12345"
```

Expected examples:

```
1 -&gt; 00001
42 -&gt; 00042
123 -&gt; 00123
9999 -&gt; 09999
12345 -&gt; 12345
```

Part 17 - Generate Employee ID

CodeHub employees have IDs like:

```
EMP00042
```

Given:

```
employee_number = "42"
```

Use `zfill()` and an f-string to generate EMP00042.

Bonus

Create:

```
STU00001
STU00002
STU00125
```

for students.

Part 18 - Build a Terminal Dashboard

Now you're going to use:

```
.center()  
.ljust()  
.rjust()
```

to create a professional-looking console interface.

Create:

```
*****  
CODEHUB DASHBOARD  
*****
```

Use:

```
"CODEHUB DASHBOARD".center(30, "**")
```

Part 19 - Create a Student Table

Create this output:

Name	City	Course
John Doe	Hyderabad	Python
Alice Smith	Chennai	React
Rahul Kumar	Delhi	Django

Use `.ljust()` and `.rjust()` to align the columns. For example:

```
print("Name".ljust(20), "City".ljust(15), "Course")
```

Important

Do **not** use formatting specifiers like:

```
:<20  
:>15
```

You haven't covered those yet. The goal is to achieve the alignment using `ljust()` and `rjust()`.

Part 20 - Student Score Display

Suppose the student scores are:

```
5  
42  
87  
100
```

Convert them into 3-character values:

```
005  
042  
087  
100
```

using:

```
.zfill(3)
```

Display:

Student Score
005
042
087
100

Final Challenge - CodeHub Data Processing System

Now combine everything you've learned.

Create a program that receives:

```
Name
Email
Phone
Student ID
Course
Address
Password
Registration Number
```

Your program should perform the following:

1. Name validation

Check `isalpha()` and `istitle()`. Remember that a name containing a space will not pass `isalpha()`, so think about how you want to handle a full name.

2. Phone validation

Use `isdigit()`.

3. Student ID validation

Use `isalnum()`.

4. Password analysis

Use `isupper()`, `islower()`, and `isalnum()`.

5. Address processing

Use `partition()`.

6. Data cleaning

Use `replace()` to clean unwanted characters.

7. Course processing

Use `splitlines()` and `join()`.

8. Registration number

Use `zfill()`.

9. Dashboard

Use `center()`, `ljust()`, and `rjust()` to create a clean terminal interface.

Expected Final Output

Your final program should produce something roughly like:

```
*****
CODEHUB DASHBOARD
*****

STUDENT INFORMATION
-----

Name           : John Doe
Email          : john.doe@gmail.com
Phone          : 9876543210
Student ID     : PYTHON2026
Course         : Python Full Stack
Registration No. : STU00042

-----
VALIDATION
-----

Name Valid     : True
```

```
Phone Valid      : True
Student ID Valid : True
Password AlphaNum: True
Name Title Case  : True
```

```
-----
                        PROFILE STATUS
-----
```

```
REGISTRATION SUCCESSFUL
```

```
*****
```

Bonus Challenge - The "Messy API Response"

Pretend your backend receives this ugly piece of data:

```
name:john doe
email: JOHN.DOE@GMAIL.COM
phone: 987-654-3210
student_id: PYTHON_2026
course: Python#Full#Stack
registration: 42
```

Your job is to transform it into a clean profile.

You should use as many of the newly learned methods as possible:

```
splitlines()
partition()
replace()
isalpha()
isdigit()
isalnum()
isspace()
isupper()
islower()
istitle()
isnumeric()
center()
ljust()
rjust()
zfill()
```

Final Output

```
=====
                        CLEAN USER PROFILE
=====
```

```
Name           : John Doe
Email           : john.doe@gmail.com
Phone           : 9876543210
Student ID      : PYTHON2026
Course          : Python Full Stack
Registration No. : 00042
```

Extra challenge: Don't just make the output look right - make the program **detect invalid data and report which fields need correction**. This is where the assignment starts feeling much more like an actual backend validation problem rather than a string-method worksheet.